

Big Data Ecosystem Rapport de Projet

Résumé

Ce rapport présente la conception et la mise en œuvre d'un pipeline de traitement par lots Big Data cohérent, dédié à l'analyse des trajets en vélopartage à New York. Le projet utilise un jeu de données CSV nettoyé. Le fichier est d'abord transféré vers le cluster Hadoop et stocké dans HDFS. Une table externe Hive est ensuite créée pour exposer le fichier brut, et une seconde table gérée est générée pour convertir les horodatages, calculer la durée des trajets et filtrer les lignes invalides. Une fois préparé, le jeu de données est interrogé avec SQL afin d'obtenir des statistiques.

1. Contexte et cas d'usage choisi

Les systèmes de vélos en libre-service génèrent quotidiennement d'importants volumes de données de déplacement. Chaque trajet laisse une trace : une heure de départ, une heure d'arrivée, une station de départ, une station d'arrivée, des coordonnées GPS et des informations contextuelles supplémentaires telles que le profil de l'utilisateur ou le type de vélo. Le vélo en libre-service constitue donc un cas d'usage pertinent et réaliste pour un projet Big Data.

Ce projet vise à répondre à la problématique suivante : comment collecter, stocker, structurer et analyser les données historiques de trajets afin de mieux comprendre l'utilisation d'un réseau de vélos en libre-service ? Les informations obtenues à l'issue d'analyses permettent d'optimiser la répartition des vélos, la dimension des stations, les opérations et la qualité du service.

- **Principales questions abordées par le pipeline :**
- Quelles sont les stations de départ les plus fréquentées ?
- Quelles sont les stations d'arrivée les plus fréquentées ? Comparaison stations de départ vs arrivée.
- Quelles sont les heures de pointe ?
- Quelle est la durée moyenne d'un trajet à vélo après suppression des enregistrements invalides ?

2. Origine et structure de la donnée

Origine des données : Le jeu de données utilisé est un fichier CSV nettoyé que nous avons appelé `bike_ny.csv`. Il contient l'historique des trajets en vélopartage à New York. Le fichier a été nettoyé localement en Python avant son importation afin de garantir la stabilité du schéma.

Format et type de données : Le fichier source est un fichier CSV. Elles sont structurées ce qui facilite leur chargement dans une table externe Hive. Les horodatages sont initialement stockés sous forme de chaînes de caractères dans la table brute afin de préserver le contenu original du fichier et d'éviter les problèmes d'analyse lors de l'importation. Ils sont ensuite convertis en valeurs d'horodatage dans la table analytique nettoyée.

Schéma du fichier CSV nettoyé

Champ	Type	Description
ride_id	STRING	Id unique de chaque course en vélo.
started_at	STRING	Heure précise du départ.
ended_at	STRING	Heure d'arrivée de la course .

start_station_name	STRING	Nom de la station de départ.
end_station_name	STRING	Nom de la station d'arrivée.
start_lat	DOUBLE	Latitude de la station de départ
start_lng	DOUBLE	Longitude de la station d'arrivée.
end_lat	DOUBLE	Latitude de la station d'arrivée.
end_lng	DOUBLE	Longitude de la station d'arrivée.
member_casual	STRING	Catégorie du cycliste pour analyse marketing
rideable_type	STRING	Type de vélo utilisé.

Préparation des données :

- Le script charge deux fichiers CSV de trajets Citi Bike puis les fusionne en un seul dataset pour travailler sur toutes les données en même temps.
- Il nettoie ensuite les lignes inutiles en supprimant celles qui sont vides, celles sans dates de début/fin et celles sans station de départ.
- Les colonnes de dates sont converties en format datetime, puis les lignes avec dates invalides sont supprimées.
- La durée du trajet est recalculée en secondes, et les valeurs incohérentes sont retirées (durées négatives ou supérieures à 24h).
- Enfin, le script supprime les doublons, affiche un rapport de nettoyage et enregistre un nouveau fichier CSV propre prêt pour l'analyse
- Nous obtenons un dataset de 1886320 lignes, ce qui est cohérent à un projet de big data.

3. Architecture big data

Mode de traitement : Le pipeline implémenté repose sur une architecture par lots (batch). Les données historiques de trajets sont collectées dans un fichier CSV, transférées vers le cluster, stockées dans HDFS, accessibles via une table Hive externe, transformées dans une table gérée propre, puis interrogées pour analyse.

Architecture globale du pipeline



Figure 1. Pipeline de traitement par lots de bout en bout mis en œuvre pour le projet de vélos en libre-service.

Utilisation des différents composants

Machine locale : Stocke le fichier CSV nettoyé avant ajout et sert à valider le fichier, à préparer le schéma et à envoyer l'ensemble de données au cluster distant.

SSH : Permet de transférer le fichier, de créer des répertoires HDFS et de lancer des commandes de validation.

HDFS : Stocke l'ensemble de données brutes dans un système de fichiers distribué, sous le répertoire du projet. HDFS constitue la couche de persistance des données historiques.

Table externe Hive : Fournit un accès SQL au fichier CSV brut sans déplacer ni réécrire le fichier d'origine.

Table de nettoyage Hive : Stocke les enregistrements transformés et filtrés dans un format plus analytique. C'est ici que sont effectués la conversion des horodatages et le calcul de la durée des trajets.

YARN : Gère les ressources du cluster lors de l'exécution de tâches Hive sur l'écosystème Hadoop.

Couche analytique SQL : Exécute des requêtes pour produire les principaux indicateurs du projet : popularité des stations, heures de pointe, durée moyenne des trajets et segmentation de l'utilisation.

4. Détails d'implémentation

4.1 HDFS

Un répertoire dédié est créé dans HDFS pour la zone brute du projet. Une fois le fichier local copié sur le nœud périphérique, il est inséré dans HDFS. Cette étape transforme le fichier CSV local en un fichier distribué utilisable par Hive.

```
hdfs dfs -mkdir -p /education/ece_2026_spring_gr01/l.chloe-ece/project_bike/raw
hdfs dfs -put -f ~/bike_ny.csv /education/ece_2026_spring_gr01/l.chloe-ece/project_bike/raw/
hdfs dfs -ls /education/ece_2026_spring_gr01/l.chloe-ece/project_bike/raw
hdfs dfs -cat /education/ece_2026_spring_gr01/l.chloe-ece/project_bike/raw/bike_ny.csv | head
```

4.2 Creation de la table brute Hive

La première table Hive est externe car le projet doit conserver le fichier brut sur HDFS. Une table externe *bike_trips_ext* lit directement le fichier CSV sans en prendre le contenu. Nous faisons cela car la couche de stockage et la couche d'exposition SQL doivent rester clairement séparées.

Tous les champs de date bruts sont d'abord chargés sous forme de chaînes de caractères. Ce choix est intentionnel : il évite les problèmes d'analyse syntaxique lors de l'ajout et garantit la cohérence de la couche brute au fichier source.

```
USE ece_2026_spring_gr01;
DROP TABLE IF EXISTS bike_trips_ext;
CREATE EXTERNAL TABLE bike_trips_ext (
  ride_id STRING,
  started_at STRING,
  ended_at STRING,
  start_station_name STRING,
  end_station_name STRING,
  start_lat DOUBLE,
  start_lng DOUBLE,
  end_lat DOUBLE,
  end_lng DOUBLE,
  member_casual STRING,
  rideable_type STRING
)
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'
WITH SERDEPROPERTIES ('separatorChar' = ',', 'quoteChar' = '"', 'escapeChar' = '\\')
STORED AS TEXTFILE
LOCATION '/education/ece_2026_spring_gr01/l.chloe-ece/project_bike/raw'
TBLPROPERTIES ('skip.header.line.count'='1');
```

4.3 Création du tableau analytique propre

Une seconde table *bike_trips_clean* est créée à partir de la table externe afin de préparer les données pour l'analyse. Les horodatages sont convertis de chaînes de caractères en valeurs numériques, la durée du trajet est calculée en minutes et les lignes invalides sont supprimées. Cette table représente la couche analytique du

projet. La logique de transformation est essentielle car les données brutes de transport peuvent contenir des noms de stations incomplets, des horodatages incorrects ou des durées incohérentes.

```
DROP TABLE IF EXISTS bike_trips_clean;
CREATE TABLE bike_trips_clean
STORED AS ORC
AS
SELECT
  ride_id,
  CAST(from_unixtime(unix_timestamp(regexp_replace(started_at, 'T', ' '), 'yyyy-MM-dd HH:mm:ss')) AS TIMESTAMP) AS
  started_at_ts,
  CAST(from_unixtime(unix_timestamp(regexp_replace(ended_at, 'T', ' '), 'yyyy-MM-dd HH:mm:ss')) AS TIMESTAMP) AS
  ended_at_ts,
  start_station_name,
  end_station_name,
  start_lat, start_lng, end_lat, end_lng,
  member_casual,
  rideable_type,
  (unix_timestamp(regexp_replace(ended_at, 'T', ' '), 'yyyy-MM-dd HH:mm:ss')
  - unix_timestamp(regexp_replace(started_at, 'T', ' '), 'yyyy-MM-dd HH:mm:ss')) / 60.0 AS trip_duration_min
FROM bike_trips_ext
WHERE ride_id IS NOT NULL
AND started_at IS NOT NULL
AND ended_at IS NOT NULL
AND start_station_name IS NOT NULL
AND end_station_name IS NOT NULL
AND unix_timestamp(regexp_replace(ended_at, 'T', ' '), 'yyyy-MM-dd HH:mm:ss')
> unix_timestamp(regexp_replace(started_at, 'T', ' '), 'yyyy-MM-dd HH:mm:ss');
```

4.4 Requêtes d’analyses

start_station_name	nb_trips
W 21 St & 6 Ave	8335
1 Ave & E 68 St	6569
8 Ave & W 31 St	6350
W 31 St & 7 Ave	6035
E 33 St & 1 Ave	5905
Ave A & E 14 St	5884
Broadway & W 25 St	5821
Lafayette St & E 8 St	5703
W 41 St & 8 Ave	5458
Broadway & W 58 St	5317

Figure 2. Résultat requête utilisation stations départ

end_station_name	nb_trips
W 21 St & 6 Ave	8309
1 Ave & E 68 St	6615
8 Ave & W 31 St	6364
W 31 St & 7 Ave	6051
E 33 St & 1 Ave	5929
Ave A & E 14 St	5905
Broadway & W 25 St	5798
Lafayette St & E 8 St	5705
W 41 St & 8 Ave	5500
University Pl & E 14 St	5364

Figure 3. Résultat requête utilisation stations arrivées

rideable_type	nb_trips
classic_bike	673039
electric_bike	1213279

Figure 5. Résultat requête types de vélos utilisés

trip_hour	nb_trips
17	165967
18	150891
16	143607
8	142193
15	131505
14	120361
9	111183
19	110102
13	107535
12	101538
11	91810
7	91266
10	88564
20	77187
21	59446
22	45632
6	43906
23	32295
0	21080
5	16144
1	13297
2	8890
4	6021
3	5898

Figure 4. Résultat requête heures de pointes

L'analyse marketing des trajets permet d'identifier les stations où il sera le plus pertinent de positionner les vélos pour le mois suivant. Les stations de départ les plus utilisées sont celles qui devront être approvisionnées en priorité 5 (voir figure 2), car elles concentrent la majorité des prises de vélos. L'étude des stations d'arrivée montre que les neuf premières stations sont également présentes parmi les stations de départ les plus fréquentées (voir figure 3). Cela signifie qu'une grande partie des vélos sera naturellement rééquilibrée par les utilisateurs eux-mêmes au fil des trajets. En revanche, la station Broadway & W 58 St devra faire l'objet d'une attention particulière, car elle pourrait nécessiter une intervention de maintenance ou un réapprovisionnement manuel si le nombre de vélos disponibles devient insuffisant.

L'analyse des heures de pointe met en évidence deux grands moments de forte activité : le matin, principalement entre 8h et 9h, puis en fin de journée, après les heures de bureau. Cela suggère que les vélos sont majoritairement utilisés pour des trajets quotidiens de type domicile-travail (voir figure 4). Cette hypothèse est renforcée par l'étude de la durée moyenne des trajets, qui est d'environ 11,02 minutes. Les usagers effectuent donc surtout des déplacements courts, probablement dans un cadre urbain fonctionnel. Enfin, l'analyse du type de vélo utilisé montre une préférence marquée pour les vélos électriques (voir figure 5). Il sera donc pertinent, dans la stratégie de placement, de prévoir une proportion plus importante de vélos électriques dans les stations les plus sollicitées.

5. Test et Validation

Nous allons maintenant réaliser des tests. La validation a été effectuée à chaque étape majeure du pipeline : stockage, ingestion Hive, transformation et analyse. L'objectif des tests n'est pas seulement de prouver qu'une commande s'exécute, mais aussi de vérifier que le résultat est cohérent avec le flux de données attendu.

- Vérifiez que le fichier CSV existe dans HDFS et qu'il est lisible depuis le cluster.

```
[l.chloe-ece@edge-1 ~]$ hdfs dfs -ls /education/ece_2026_spring_gr01/l.chloe-ece/project_bike/raw
Found 1 items
-rw-r--r--  2 l.chloe-ece ece_2026_spring_gr01 338604326 2026-03-24 13:10 /education/ece_2026_spring_gr01/l.chloe-ece/project_bike/raw/bike_ny.csv
[l.chloe-ece@edge-1 ~]$ hdfs dfs -du -h /education/ece_2026_spring_gr01/l.chloe-ece/project_bike/raw
322.9 M 645.8 M /education/ece_2026_spring_gr01/l.chloe-ece/project_bike/raw/bike_ny.csv
```

- Vérifiez que la table externe affiche les lignes attendues, sans en-tête, et les types de données corrects.

```
0: jdbc:hive2://zoo-1.au.adaltas.cloud:2181,2> DESCRIBE bike_trips_ext;
INFO : Compiling command(queryId=hive_20260325153424_83594003-bde4-4490-a795-cd9b41b1d23): DESCRIBE bike_trips_ext
INFO : Semantic Analysis Completed (retrial = false)
INFO : Returning Hive schema: Schema(FieldSchemas:[FieldSchema(name:col_name, type:string, comment:from deserializer), FieldSchema(name:data_type, type:string, comment:from deserializer), FieldSchema(name:comment, type:string, comment:from deserializer)], properties:null)
INFO : Completed compiling command(queryId=hive_20260325153424_83594003-bde4-4490-a795-cd9b41b1d23); Time taken: 0.071 seconds
INFO : Executing command(queryId=hive_20260325153424_83594003-bde4-4490-a795-cd9b41b1d23): DESCRIBE bike_trips_ext
INFO : Starting task [Stage=0:DOL] in serial mode
INFO : Completed executing command(queryId=hive_20260325153424_83594003-bde4-4490-a795-cd9b41b1d23); Time taken: 0.048 seconds
INFO : OK
```

col_name	data_type	comment
ride_id	string	from deserializer
started_at	string	from deserializer
ended_at	string	from deserializer
start_station_name	string	from deserializer
end_station_name	string	from deserializer
start_lng	string	from deserializer
end_lng	string	from deserializer
member_casual	string	from deserializer
rideable_type	string	from deserializer

11 rows selected (0.534 seconds)

- Vérifiez que la table nettoyée contient des horodatages analysés et un champ de durée valide.

ride_id	started_at_ts	ended_at_ts	trip_duration_min
8E865410DBDE0CA9	2024-01-01 13:00:04.0	2024-01-01 13:04:04.0	4.000000
0403D0B3FC9CA77D	2024-01-08 19:36:43.0	2024-01-08 19:53:16.0	16.550000
F6DE7BB42FF550BE	2024-01-12 15:00:41.0	2024-01-12 15:36:29.0	35.800000
84A995BFD98030D4	2024-01-12 16:52:19.0	2024-01-12 17:17:29.0	25.166667
7BBEAD4F2B535813	2024-01-05 19:50:19.0	2024-01-05 20:34:42.0	44.383333
296A378A5C42028D	2024-01-03 17:45:12.0	2024-01-03 18:16:03.0	30.850000
9A891F9C924C240F	2024-01-05 09:47:22.0	2024-01-05 09:50:49.0	3.450000
219B18A37682C36	2024-01-06 08:35:40.0	2024-01-06 08:48:30.0	12.833333
9EA1D9C604956D7D	2024-01-12 07:59:57.0	2024-01-12 08:10:05.0	10.133333
568F62EE667D0E39	2024-01-01 18:08:59.0	2024-01-01 18:13:09.0	4.166667

- Vérifiez que le nombre de lignes reste cohérent après transformation et que les enregistrements invalides sont supprimés : on a le même nombre car on avait déjà pré-nettoyé les données

```
+-----+
| nb_rows_ext |
+-----+
| 1886318    |
+-----+
```

```
+-----+
| nb_rows_clean |
+-----+
| 1886318      |
+-----+
```

Pour être sûrs qu'il n'y a aucune valeurs nulles dans notre dataset, on exécute :

```
SELECT COUNT(*) AS nb_invalid_remaining
FROM bike_trips_clean
WHERE ride_id IS NULL
OR started_at_ts IS NULL
OR ended_at_ts IS NULL
OR start_station_name IS NULL
OR end_station_name IS NULL
OR trip_duration_min <= 0;
```

```
+-----+
| nb_invalid_remaining |
+-----+
| 0                    |
+-----+
```

6. Bonus : passer en mode hybride et simuler des données en temps réel

Dans cette partie bonus, nous avons étendu notre pipeline batch initial vers un fonctionnement hybride. En plus du fichier CSV historique déjà chargé dans HDFS et analysé avec Hive, nous avons ajouté un second fichier CSV représentant de nouvelles données arrivants plus tard, afin de simuler un ajout de données en quasi temps réel. Ce second fichier a été stocké dans un dossier HDFS séparé, puis lu à l'aide d'une nouvelle table externe Hive appelée *bike_trips_live_ext*.

Nous avons ensuite appliqué le même processus de nettoyage et de transformation que pour les données historiques, afin d'obtenir une table propre contenant des horodatages correctement interprétés et une durée de trajet calculée, et avons obtenu la table *bike_trips_live_clean*.

Enfin, nous avons fusionné les données historiques et les nouvelles données dans une vue hybride unique, dans *bike_trips_hybrid*. Cela nous permet d'analyser à la fois l'historique et les nouvelles arrivées dans une seule structure, tout en montrant qu'il est possible de faire évoluer une architecture batch vers une architecture hybride plus proche d'un fonctionnement réel.

```
+-----+
| data_source | nb_rows |
+-----+
| historical_batch | 1886318 |
| simulated_live   | 1997530 |
+-----+
```

```
+-----+
| nb_rows_hybrid |
+-----+
| 3883848        |
+-----+
```

```
+-----+
| start_station_name | nb_trips |
+-----+
| W 21 St & 6 Ave    | 17291    |
| 1 Ave & E 68 St    | 13492    |
| 8 Ave & W 31 St    | 13400    |
| Lafayette St & E 8 St | 12630    |
| W 31 St & 7 Ave    | 12301    |
| Broadway & W 25 St | 12266    |
| Ave A & E 14 St    | 11989    |
| E 33 St & 1 Ave    | 11452    |
| W 41 St & 8 Ave    | 11367    |
| Broadway & W 58 St | 11065    |
+-----+
```

Les vérifications réalisées montrent que la vue hybride fonctionne correctement. Tout d'abord, nous avons confirmé que les deux sources de données, historiques et simulées, étaient bien présentes individuellement dans les tables nettoyées. Ensuite, le comptage total des lignes dans la vue hybride a permis de vérifier que celle-ci regroupe bien l'ensemble des données issues des deux sources. La requête de répartition par *data_source* prouve également que la fusion a bien conservé l'origine des données, avec une partie provenant du traitement batch historique et une autre du flux simulé. Enfin, une requête d'analyse exécutée directement sur la vue hybride montre que cette dernière est pleinement exploitable pour produire des résultats métier cohérents, ce qui valide le bon fonctionnement de l'architecture hybride mise en place.